

MC60 OpenCPU

GS3 SDK — First Test Report

SDK	MC60_OpenCPU_GS3_SDK_V1.8
Toolchain	Arm GNU Toolchain 15.2.1
Host OS	Windows 11
Date	June 2026
Example	GPS (GNSS)
Interface	PuTTY via CP2102 (COM6, 115200 baud)

Contents

1	Introduction	2
2	Toolchain Installation	2
2.1	Why a Cross-Compiler is Needed	2
2.2	Arm GNU Toolchain 15.2.1	2
3	CP2102 USB-to-UART Driver	3
3.1	Problem	3
3.2	Fix	3
4	Build Configuration	3
4.1	Enabling the GPS Example	4
4.2	Other Build Fixes	4
5	Flashing with QFlash	4
6	Serial Terminal Setup (PuTTY)	4
7	GNSS Output and Observations	5
7.1	Terminal Output	5
7.2	NMEA Sentence Analysis	6
7.3	Assessment	6
8	Summary	7
	References	7

1. Introduction

This report documents the first end-to-end test of the **Quectel MC60** cellular/GNSS module using the *MC60_OpenCPU_GS3_SDK_V1.8*. The objective was to compile a GPS example from source, flash it to the module, and verify GNSS output over a serial terminal.

The MC60 is an ARM Cortex-based module, so the standard host GCC (which targets x86) cannot be used directly. A dedicated **ARM cross-compilation toolchain** was required. The SDK itself was written circa 2012–2013 and originally targeted GCC 4.7.2 (CodeSourcery toolchain); bringing it up to build correctly on a modern Windows 11 machine with GCC 15.2.1 required a series of targeted fixes to the makefiles, which are documented in a companion document (*MC60_Build_Changes.pdf*).

This report focuses on the overall workflow: toolchain installation, the build process, USB driver setup, serial terminal configuration, and the GNSS output observed.

2. Toolchain Installation

2.1. Why a Cross-Compiler is Needed

The MC60 module is based on an ARM core (MediaTek MT6261). A standard GCC installation on a Windows or Linux PC targets the host architecture (x86-64) and cannot produce binaries that run on ARM. A **cross-compiler** is a compiler that runs on one architecture (the host) but generates machine code for a different architecture (the target).

The required toolchain is **arm-none-eabi-gcc**: it runs on Windows x86 and produces ARM bare-metal binaries, which is exactly what the MC60 needs.

2.2. Arm GNU Toolchain 15.2.1

The original SDK targeted CodeSourcery Sourcery CodeBench Lite, a 2012-era toolchain that no longer installs correctly on Windows 11. The modern replacement is the **Arm GNU Toolchain**, distributed directly by Arm.

After downloading and installing the `mingw-w64-i686-arm-none-eabi` variant, the toolchain binary directory was added to the system PATH:

Listing 1: Adding the Arm GNU Toolchain to the system PATH (PowerShell)

```
[System.Environment]::SetEnvironmentVariable(  
    "Path",  
    $env:Path + ";C:\Program Files (x86)\Arm\GNU Toolchain mingw-w64-i686-arm-none-  
    eabi\bin",  
    [System.EnvironmentVariableTarget]::Machine  
)
```

The makefile was then updated to point to the new installation:

Listing 2: Updated toolchain path and version in `make/gcc/gcc_makefile`

```
GCC_INSTALL_PATH=C:\Program Files (x86)\Arm\GNU Toolchain mingw-w64-i686-arm-none-eabi
GCC_VERSION=15.2.1
```

Note on toolchain compatibility

The SDK makefile was written for GCC 4.7.2 (2012). GCC 15 introduces several breaking changes: warnings that were previously silent are now hard errors, library paths have changed, and certain linker flags accepted by the old compiler driver are rejected by the modern linker. These issues were resolved through targeted makefile edits documented in the companion build-fix document.

3. CP2102 USB-to-UART Driver

3.1. Problem

The MC60 EVB connects to the PC via a **Silicon Labs CP2102** USB-to-UART bridge chip. On a fresh Windows 11 installation the VCP (Virtual COM Port) driver is not present, so the device appears under *Other Devices* in Device Manager with a warning triangle rather than as a usable COM port (Figure 1).

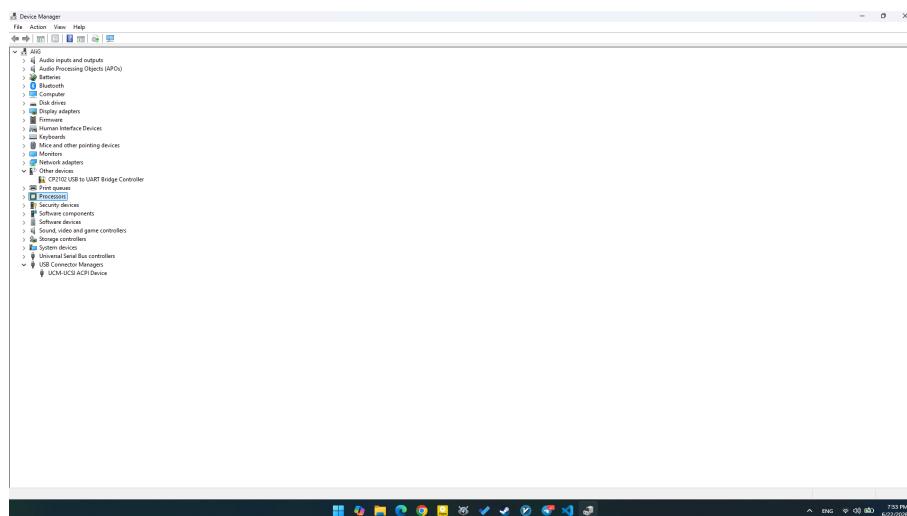


Figure 1: Device Manager showing the CP2102 listed under *Other Devices* (no driver installed).

3.2. Fix

The **CP210x VCP driver** was downloaded from the Silicon Labs website and installed. After unplugging and replugging the EVB, Windows assigned it **COM6** (Silicon Labs CP210x USB to UART Bridge), confirming the driver was working correctly.

4. Build Configuration

4.1. Enabling the GPS Example

All example applications in the SDK are guarded by preprocessor defines (`#ifdef __EXAMPLE_XXX__`). Without the corresponding define, the source file compiles to effectively empty code. The GPS example was enabled by adding the flag to the makefile:

Listing 3: Enabling the GPS example via preprocessor define

```
C_PREDEF=-D __CUSTOMER_CODE__ -D __EXAMPLE_GPS__
```

4.2. Other Build Fixes

Beyond the toolchain path and GPS enable flag, additional makefile changes were necessary to handle differences between GCC 4.7.2 and GCC 15. These included updating library paths, removing a CodeSourcery-only file deletion utility, removing an invalid linker flag, and suppressing type-safety warnings that GCC 14+ promotes to hard errors. Full details are given in the companion document *MC60_Build_Changes.pdf*.

5. Flashing with QFlash

After a successful build, the firmware binary was flashed to the MC60 using **QFlash V4.0**. The tool requires three file references: a scatter file (**SCAT**), a download agent (**DA**), and the compiled application binary (**APPGS3MDM32A01.bin**). Figure 2 shows the QFlash configuration used.

name	Location
☒ SCAT	C:\Users\Ali\Downloads\Compressed\SDK\MC60_OpenCPU_GS3_SDK_V1.8\build\gcc\app_image_bin.cfg
☒ DA	C:\Users\Ali\Downloads\Compressed\SDK\MC60_OpenCPU_GS3_SDK_V1.8\tools\QFlash_V4.0\MTK_AllInO...
☒ APPGS3MDM32A01.bin	C:\Users\Ali\Downloads\Compressed\SDK\MC60_OpenCPU_GS3_SDK_V1.8\build\gcc\APPGS3MDM32A01.bin

Figure 2: QFlash V4.0 download configuration with the correct MC60 scatter file, DA, and compiled binary.

6. Serial Terminal Setup (PuTTY)

Serial output from the module was monitored using **PuTTY** in Serial mode. The connection parameters are shown in Figure 3.

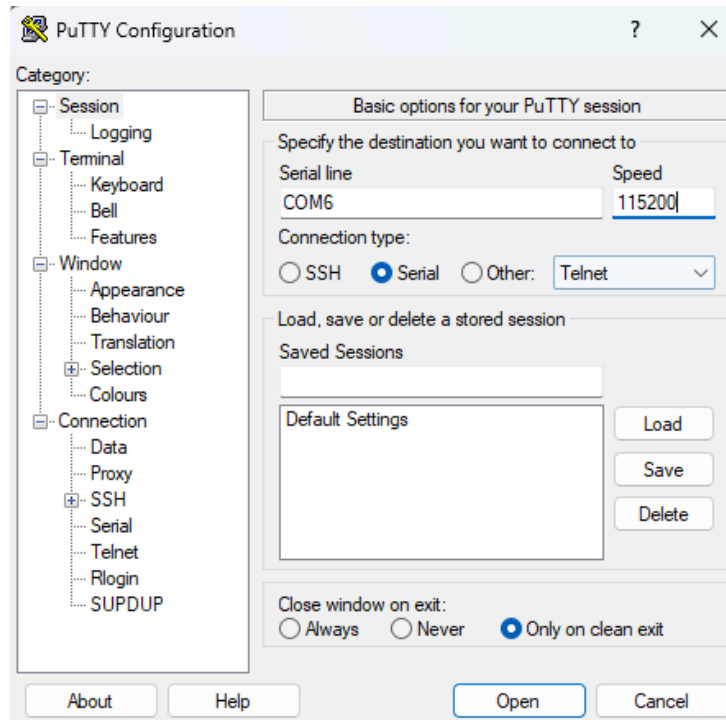


Figure 3: PuTTY configuration: Serial, COM6, 115200 baud.

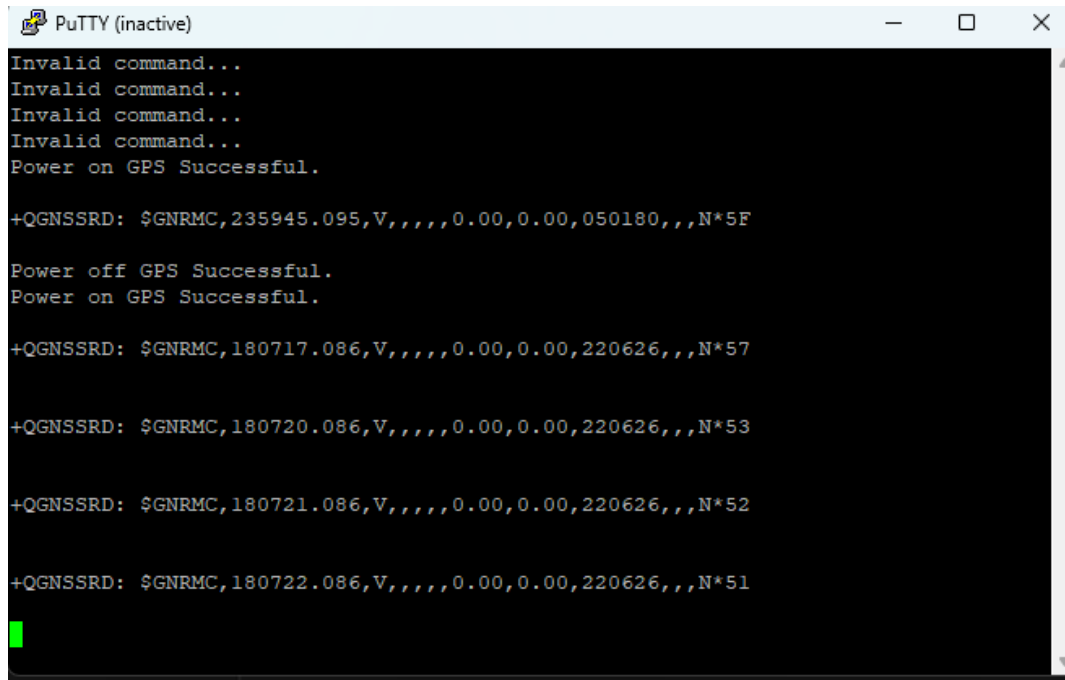
PuTTY Connection Parameters

Serial line	COM6
Speed	115200 baud
Connection type	Serial

7. GNSS Output and Observations

7.1. Terminal Output

With the firmware flashed and PuTTY connected, the module was powered on. The GPS example executed and the GNSS subsystem began emitting NMEA sentences over the UART. Figure 4 shows the terminal output captured during the test.



```
PuTTY (inactive)
Invalid command...
Invalid command...
Invalid command...
Invalid command...
Power on GPS Successful.

+QGNSSRD: $GNRMC,235945.095,V,,,,,0.00,0.00,050180,,N*5F

Power off GPS Successful.
Power on GPS Successful.

+QGNSSRD: $GNRMC,180717.086,V,,,,,0.00,0.00,220626,,N*57

+QGNSSRD: $GNRMC,180720.086,V,,,,,0.00,0.00,220626,,N*53

+QGNSSRD: $GNRMC,180721.086,V,,,,,0.00,0.00,220626,,N*52

+QGNSSRD: $GNRMC,180722.086,V,,,,,0.00,0.00,220626,,N*51

█
```

Figure 4: PuTTY terminal showing GNSS NMEA output (+QGNSSRD: \$GNRMC) from the MC60.

7.2. NMEA Sentence Analysis

The module outputs +QGNSSRD URC (Unsolicited Result Code) messages wrapping standard NMEA \$GNRMC sentences. The observed output is of the form:

Listing 4: Representative GNSS output from the MC60

```
+QGNSSRD: $GNRMC,180717.086,V,,,0.00,0.00,220626,,,N*57
+QGNSSRD: $GNRMC,180720.086,V,,,0.00,0.00,220626,,,N*53
+QGNSSRD: $GNRMC,180721.086,V,,,0.00,0.00,220626,,,N*52
```

The key field in each sentence is the **status character**: **V** (“Void”) instead of **A** (“Active”). A status of **V** indicates that the GNSS receiver has not yet obtained a valid fix — the coordinate fields are consequently empty.

7.3. Assessment

The module is **operational** and the firmware is executing correctly: the GNSS subsystem is powered, the receiver is scanning for satellites, and NMEA sentences are being generated and forwarded over the UART at the expected rate. The absence of a fix at this stage is expected and is attributable to one or more of the following:

- **Indoor / obstructed sky view** — the module was tested indoors with no clear line of sight to the sky.
- **Cold start** — the receiver has no almanac or ephemeris data cached, so Time-To-First-Fix (TTFF) is typically 30–90 seconds outdoors and much longer or indefinite indoors.

- **Antenna** — the EVB passive patch antenna may have insufficient gain in the test environment.
- **Supply voltage** — the GNSS front-end is sensitive to supply noise; an inadequate or noisy supply can reduce receiver sensitivity below the threshold needed to acquire satellites.

8. Summary

Table 1 summarises the steps completed and the outcome of each.

Table 1: Test summary

Step	Notes	Status
Arm GNU Toolchain 15.2.1 install	Installed arm-none-eabi cross-compiler; added to PATH	Done
Makefile fixes	Toolchain path, GPS define, library paths, linker flags, warning suppression	Done
CP2102 VCP driver	Installed Silicon Labs driver; EVB enumerated as COM6	Done
Build (GPS example)	Compiled successfully with GCC 15.2.1	Done
Flash via QFlash	Binary flashed with correct MC60 scatter and DA files	Done
PuTTY serial monitor	Connected at 115200 baud on COM6	Done
GNSS output	\$GNRMC sentences received; no fix yet (V status)	No Fix

The overall first test is considered **successful**: the cross-compilation toolchain is set up, the SDK builds cleanly, the firmware runs on the module, and GNSS output is visible over the serial interface. Obtaining an actual GNSS fix requires moving the hardware outdoors with a clear sky view, or waiting for a longer cold-start acquisition time.

References

- [1] Quectel, *MC60 OpenCPU GS3 SDK V1.8*, SDK release package.
- [2] Arm Ltd., *Arm GNU Toolchain 15.2.1 (mingw-w64-i686-arm-none-eabi)*, <https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads>.
- [3] Silicon Labs, *CP210x USB to UART Bridge VCP Drivers*, <https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers>.
- [4] NMEA 0183 Standard, National Marine Electronics Association, 2023.
- [5] *MC60_Build_Changes.pdf* — companion makefile fix documentation, June 2026.